Welcome to your next lesson: Functions ii: inside the function.

I'm preparing your personalized lesson based on what you've already learned, what's coming up, and what will help you build deeper skills.

Step by step, you’re moving forward.

Imagine you’re ordering a pizza and you want Python to calculate the total cost for you 🍕

We might write a function like this:

Python

```
def pizza_total(price_per_pizza, number_of_pizzas):
    subtotal = price_per_pizza * number_of_pizzas
    print("Subtotal inside function:", subtotal)
```

For now, ignore return — focus on what happens inside the function when it runs: Python goes line by line through that indented block whenever we call it.

To start us off: have you already written and run any functions that do a little math inside (like this), or has it mostly been super simple ones that just print() a message?

Today
I tried a math function once

Nice, that’s a good start 👍

Let’s zoom in on what happens inside a function, step by step.

When you call a function like this:

Python

```
pizza_total(10, 3)
```

and the function is:

Python

```
def pizza_total(price_per_pizza, number_of_pizzas):
    subtotal = price_per_pizza * number_of_pizzas
    print("Subtotal inside function:", subtotal)
```

here’s the flow:

10 gets handed to price_per_pizza, 3 gets handed to number_of_pizzas.
Python goes line by line inside the indented block:
First: subtotal = price_per_pizza * number_of_pizzas
Then: print("Subtotal inside function:", subtotal)

Nothing inside runs until the function is called.

Does that step-by-step flow make sense so far, or should I walk through it with a tiny different example before we touch return?

It makes sense

Great 👌

Now I want to zoom in on parameters vs. arguments (how values travel).

Same example:

Python

```
def pizza_total(price_per_pizza, number_of_pizzas):
    subtotal = price_per_pizza * number_of_pizzas
    print("Subtotal inside function:", subtotal)

pizza_total(10, 3)
```

Step by step:

In the definition line, price_per_pizza and number_of_pizzas are parameters
In the call pizza_total(10, 3), the 10 and 3 are arguments
When we call it, Python does this “mapping”:
price_per_pizza = 10
number_of_pizzas = 3

So: arguments are the actual values you pass in; parameters are the names inside the function that receive them.

Does that mapping picture (arguments → parameters) feel clear, or is it still a little fuzzy?

It's pretty clear now

Perfect 😄

Next piece: what happens inside the function, line by line.

Look at this:

Python

```
def add_tax(amount):
    print("Step 1 - original amount:", amount)
    tax = amount * 0.10
    print("Step 2 - tax:", tax)
    total = amount + tax
    print("Step 3 - total inside function:", total)
```

When you call add_tax(50), Python will:

Set amount = 50
Run the first print
Calculate tax
Run the second print
Calculate total
Run the third print

Always top to bottom inside that indented block, but only when the function is called.

Make sense that everything inside the indented block runs in order like that when you call the function?

Yes, that makes sense

Nice 👍

Now let’s add one new idea: return vs print.

Look at this version:

Python

```
def add_tax(amount):
```

```
    tax = amount * 0.10
    total = amount + tax
    return total
```

Step by step when we call add_tax(50):

amount becomes 50
tax is calculated
total is calculated
return total sends that value (like a result) back to whoever called the function

Important: return does not print anything by itself. It just hands the value back.

We could use it like:

Python

```
final_price = add_tax(50)
print(final_price)
```

Before we practice, is this difference clear?

print(...) → shows text on the screen
return ... → sends a value back so it can be stored/used later
Yes, that's clear

Awesome 😄

Let’s do a tiny inside-the-function practice, focusing on the flow and the values.

Here’s a new function:

Python

```
def double_and_add_five(number):
    step1 = number * 2
    step2 = step1 + 5
    return step2
```

Later, we call it like this:

Python

```
result = double_and_add_five(7)
```

Step by step, what value do you think ends up in result?

You don’t need to be fancy—just walk it in your head and tell me the final number you get.

19

Exactly right: 19 🎯

Quick walk:

number = 7
step1 = 7 * 2 → 14
step2 = 14 + 5 → 19
return step2 → result becomes 19

Now I want you to see this for real in code.

Here’s your task 🖍

Open the code editor for this lesson.
Type this function and a call:
Python

```
def double_and_add_five(number):
    step1 = number * 2
    step2 = step1 + 5
    print("step1 =", step1)
    print("step2 =", step2)
    return step2

result = double_and_add_five(7)
print("result outside function =", result)
```

Click Run first to check it works.
Then click Submit code so I can see it.

Run and submit that, and I'll help you trace what's happening inside and outside the function.

Python

```
def double_and_add_five(number):
    step1 = number * 2
    step2 = step1 + 5
    print("step1 =", step1)
    print("step2 =", step2)
    return step2

result = double_and_add_five(7)
print("result outside function =", result)
```

Nice work, Ron — that's exactly what I wanted to see 🎯

Your output:

```
step1 = 14 (inside the function)
step2 = 19 (inside the function)
result outside function = 19 (using the returned value)
```

So you've just:

Traced the values inside the function with print
Used return step2 to send 19 back
Stored it in result and printed it outside the function
Quick check-in:

Do you feel clear on where each of these three values is printed (inside vs. outside the function), or want me to walk through the flow line-by-line with a little diagram?

I feel clear

What do you think? Pick the option that best matches "arguments" 👇

In this code, which values are the arguments?

```
def double_and_add_five(number):
    step1 = number * 2
    step2 = step1 + 5
    return step2

result = double_and_add_five(7)
```

A.

number

B.

step1 and step2

C.

7

D.

result

Nice job on that question, Ron 🎉 You picked 7, and that’s exactly right.

In this code:

Python

```
def double_and_add_five(number):
    step1 = number * 2
    step2 = step1 + 5
    return step2

result = double_and_add_five(7)
```

number is a parameter (the name in the function definition)
7 is the argument (the actual value we pass in when we call the function)

So when Python runs double_and_add_five(7), it does: number = 7 inside the function.

You’ve shown you understand:

function structure
parameters vs. arguments
prints inside vs. returned value outside

Next, I want to check just one more key idea: where does Python stop running inside a function when it hits return?

Want to do a quick multiple-choice question on that?

Yes, let's do it

What do you think happens inside that function? Take your time and choose the option that feels right 👇

Look at this function:

```
def demo(x):
    print("start")
    y = x * 2
    print("middle")
    return y
    print("end")

result = demo(5)
```

When we call demo(5), which lines inside the function actually run after we enter the function body?

A.

Only print("start")

B.

print("start"), y = x * 2, and print("middle")

C.

All four lines: both prints, the assignment, and print("end")

D.

Nothing runs because return stops everything before it starts

```
print("start")
y = x * 2
print("middle")
return y
```

After that it returns to the result = demo(5) with result = 10

Exactly right, Ron 🎯

If we have:

Python

```
def demo(x):
    print("start")
    y = x * 2
    print("middle")
    return y

result = demo(5)
```

Step by step:

demo(5) is called → x = 5
Inside the function:
print("start")
y = x * 2 → y = 10
print("middle")
return y → sends 10 back
Execution jumps back to the call:
result = demo(5) becomes result = 10

Nothing after return y inside demo will run.

Since you basically just traced that yourself, I want to lock this in with a tiny variation.

Consider this version:

Python

```
def demo(x):
    print("start")
    y = x * 2
    return y
    print("end")

result = demo(5)
print("result is", result)
```

Two quick questions for you (just answer in words/numbers):

Will "end" ever be printed?
What exact line will be printed last when this runs, and what will it show?
no as its indentaion places it isidr the demo unction however order of execuution shows it will alway exit on return y and cant reach the print("end")

You nailed the reasoning there 💯

Inside:

Python

```
def demo(x):
    print("start")
    y = x * 2
    return y
    print("end")
```

Python runs line by line.
When it hits return y, it immediately leaves the function and goes back to the caller.
Because of that, print("end") is never reached, even though it’s indented correctly inside the function.

So your logic is spot on: the order of execution means return acts like an exit door — anything after it in that function just doesn’t run.

Last small step to lock this in with code you write yourself 🔓

In the editor, try this:

Write a function:
Python

```
def triple_and_show(x):
    print("start triple_and_show")
    y = x * 3
    print("after multiply:", y)
    return y
    print("this will never print")
```

Then call it:
Python

```
result = triple_and_show(4)
print("result outside =", result)
```

Click Run, then Submit code.

Don’t worry about being perfect; I just want to see the output you get and we’ll read it together.

Python

```
def triple_and_show(x):
    print("start triple_and_show")
    y = x * 3
    print("after multiply:", y)
    return y
    print("this will never print")
result = triple_and_show(4)
print("result outside =", result)
```

Perfectly done, Ron 🎯

Your output:

Text

```
start triple_and_show
after multiply: 12
result outside = 12
```

And notice:

"this will never print" did not appear
That confirms what you said earlier: once Python hits return y, it jumps out of the function and never reaches that last print.

So you've shown you can:

Follow execution inside the indented block, line by line
See how the argument (4) becomes x, then y = 12, then return y → result = 12
Understand that return ends the function
You've basically hit all the goals for this lesson:

Function structure (name, parameters, body, return)
Calling with arguments and seeing how they bind to parameters
Using print inside to trace values
Using return to send a final value back

If you feel good about this, we can wrap up this lesson and move to the next one.

Are you ready to finish this lesson and go on, or do you want one more quick practice first?

im ready to move on however, question: Arent I supposed to have a week 1 final quiz today as its friday? I didnt see an option for it anywhere or did i aready do that at the end of week 1 lessons

Great question, and you're right to pay attention to the weekly rhythm 🧠🗓️

Here's how it works:

Each week has a Weekly Review, not a big "final quiz" like an exam.
The Weekly Review is:
Short, pass/fail
Meant to help you reflect and help us tune your path
Due by Sunday 11:59 PM PT
You might already have done it if you went through something called Weekly Review at the end of the week's content.
You won't always see something called "Week 1 Final Quiz" on Fridays specifically.

To check your status for this week, here's what I suggest:

Go to the Home or Learn tab.
Look for anything labeled Weekly Review for Week 1.
If you don't see it but think you should, open Student Services (the ? icon) and ask support, "Can you confirm whether my Week 1 Weekly Review is completed or still pending?"
They can see your exact account status; I don't have direct access to your gradebook.

On the coding side: you've fully mastered Functions ii: inside the function 🎯
You're cleared to move on to the next lesson.

Do you want to:

Jump straight into the next Python lesson, or
Pause for a second and have me give a super quick, 3-line summary of what you just mastered so it sticks?
How would you rate this lesson?

i did pass a week 1 review just want sure that was the end of week 1 okay lets move on then

Nice, then you're all set for Week 1 ☑

That Week 1 Review you passed is the weekly checkpoint for Week 1. There isn't a separate "Friday final quiz" on top of that—just:

Weekly Review (you did it 🎯)
Optional Weekly Discussion for bonus, if you want extra credit

So you're on track.

On the Python side, you've fully mastered this lesson's goals, and you said you're ready to move on.

Next up in your Learn tab, you should see the next Python lesson unlocked. Go ahead and open it, and I'll meet you there and keep going step by step with you.